

Distributed Systems Assignment

Student Name: *ROTICH DERICK KEMBOI*

Registration Number: *INTE/MK/3212/09/24*

Course: *DITRIBUTED SYSTEM*

1. Justify why edge computing is considered a form of client processing in a client-server architecture.

Edge computing is considered a form of client processing because some of the data processing is performed close to the client device instead of sending all data to a centralized server. In a traditional client-server architecture, the client sends requests to a server for processing. In edge computing, edge devices or edge servers located near the client perform part of the processing before communicating with the central server or cloud.

This approach reduces network latency, decreases bandwidth usage, improves response times, and allows applications to continue operating even when connectivity to the cloud is limited. The cloud still provides centralized storage, analytics, and management, while the edge handles time-sensitive tasks near the user. Therefore, edge computing extends rather than replaces the client-server model by moving some server-side processing closer to the client.

2. Distinguish between edge computing and cloud computing.

Edge Computing	Cloud Computing
Data is processed close to where it is generated.	Data is processed in centralized data centers.
Provides very low latency.	Latency is generally higher because data travels farther.
Suitable for real-time applications such as autonomous vehicles, IoT, and industrial automation.	Suitable for large-scale data storage, analytics, and application hosting.
Reduces bandwidth usage by processing data locally.	Requires sending data across the internet to cloud servers.
Can continue operating even with limited internet connectivity	Usually depends on reliable internet connectivity.
Has limited computing resources at edge devices.	Offers virtually unlimited computing and storage resources.

Case Scenario 1

Question

Your team is building a distributed application where different services, written in various programming languages (Python, Java, and Node.js), need to communicate with each other. What type of middleware would you choose, and what are the key considerations?

Answer

I would choose **Remote Procedure Call (RPC) middleware**, specifically **gRPC**, because it is designed for communication between services developed in different programming languages. gRPC uses Protocol Buffers (Protobuf) to define service interfaces, allowing Python, Java, and Node.js applications to communicate efficiently.

For asynchronous communication between services, I would also use a **message broker** such as RabbitMQ or Apache Kafka to improve scalability and reliability.

Key considerations

- Cross-language compatibility
- High performance and low latency
- Scalability
- Reliability and fault tolerance
- Security through encryption and authentication
- Ease of deployment and maintenance
- Support for synchronous and asynchronous communication

Using gRPC together with a message broker creates a flexible and scalable distributed system capable of supporting services written in multiple programming languages.

Case Scenario 2

Question

Your e-commerce application is experiencing significant growth. The current monolithic architecture struggles during peak shopping hours. How would you migrate to a distributed architecture?

Answer

To improve scalability and reliability, I would gradually migrate the monolithic application to a **microservices architecture**.

Step 1: Identify Business Services

Split the application into independent services such as:

- User Management
- Product Catalog
- Shopping Cart
- Inventory
- Payment
- Order Processing

- Notification Service

Step 2: Deploy Services Independently

Each service should have its own codebase and database where appropriate, allowing independent development, deployment, and scaling.

Step 3: Introduce an API Gateway

Use an API Gateway to provide a single entry point for all client requests and route traffic to the appropriate microservice.

Step 4: Use Containerization

Package each microservice using Docker containers and deploy them using Kubernetes or another orchestration platform for automatic scaling and recovery.

Step 5: Enable Service Communication

Use REST APIs or gRPC for synchronous communication and RabbitMQ or Apache Kafka for asynchronous messaging between services.

Step 6: Improve Reliability

Implement load balancing, health checks, monitoring, centralized logging, and automatic failover to ensure high availability.

Step 7: Continuous Deployment

Adopt CI/CD pipelines so that each micro service can be updated independently without affecting the entire application.

Benefits

- Improved scalability
- Better fault isolation
- Faster deployments
- Higher availability
- Easier maintenance
- Better performance during peak shopping periods

Migrating incrementally from a monolithic architecture to micro services reduces risk while allowing the application to handle increased traffic more efficiently.